Документация модуля парсинга данных

Обработка файлов срабатываний каналов

Версия 2.0 (с быстрыми функциями подсчета)

17 февраля 2026 г.

Содержание

1 Общее описание **3**

1.1 Формат данных . . . 3

1.2 Два подхода к обработке . . . 3

2 Быстрые функции (оптимизированные для производительности) **3**

2.1 count_channel_activations_fast . . . 3

2.2 get_channel_stats_fast . . . 4

2.3 count_multiple_channels_fast . . . 5

2.4 get_channel_activation_timeline_fast . . . 5

3 Стандартные функции (полный парсинг) **6**

3.1 parse_data_file . . . 6

3.2 get_events_by_timestamp . . . 7

3.3 get_channel_statistics . . . 7

3.4 print_channel_statistics . . . 7

3.5 filter_events_by_time_range . . . 8

3.6 get_cell_activation_timeline . . . 8

3.7 get_channel_heatmap . . . 9

4 Функция сравнения производительности **9**

4.1 compare_performance . . . 9

5 Сравнение методов **10**

6 Константы и настройки **10**

6.1 Формат упаковки данных . . . 10

6.2 Размер записи . . . 10

7 Полные примеры использования **10**

7.1 Пример 1: Быстрый анализ большого файла . . . 10

7.2 Пример 2: Детальный анализ с фильтрацией . . . 11

7.3 Пример 3: Сравнение производительности . . . 12

8 Рекомендации по использованию **13**

9 Обработка ошибок **13**

10 Примечания **14**

1 Общее описание

Модуль предназначен для парсинга бинарных файлов, содержащих данные о срабатываниях 4 каналов по 16 ячеек в каждом. Данные записаны в hex-формате и организованы в виде последовательных записей фиксированной длины.

1.1 Формат данных

Каждая запись в файле имеет структуру, представленную в таблице 1.

Таблица 1: Структура записи данных

Поле	**Размер (байт)**	**Описание**
Конец временной метки	2	Первая часть временной метки
Маркер 0x00BE	2	Маркер начала данных
Канал 1	2	16-битное число, биты соответствуют ячейкам 0-15
Канал 2	2	16-битное число для второго канала
Канал 3	2	16-битное число для третьего канала
Канал 4	2	16-битное число для четвертого канала
Конец временной метки	2	Вторая часть временной метки (используется как основная)
Маркер 0x00FE	2	Маркер конца данных

Общий размер одной записи составляет 16 байт.

1.2 Два подхода к обработке

Модуль предоставляет два набора функций для работы с данными:

1. **Стандартные функции** – выполняют полный парсинг файла с созданием списка всех событий. Подходят для гибкой работы с данными, фильтрации по времени и комплексного анализа.

2. **Быстрые функции** – выполняют прямой подсчет без создания промежуточных структур данных. Оптимальны для больших файлов и задач, требующих только статистики.

2 Быстрые функции (оптимизированные для производительности)

2.1 count_channel_activations_fast

```
def count_channel_activations_fast(filename: str, channel_num: int) ->
    int:
```

Листинг 1: Сигнатура функции count_channel_activations_fast

Назначение: Быстрый подсчет общего количества срабатываний в указанном канале. Не создает промежуточных структур данных, минимально использует память.

Параметры:

- `filename: str` – путь к файлу для анализа
- `channel_num: int` – номер канала (0-3)

Возвращаемое значение:

- `int` – общее количество срабатываний в канале

Исключения:

- `ValueError` – если номер канала вне диапазона 0-3
- `FileNotFoundError` – если файл не найден

Пример использования:

```
total = count_channel_activations_fast("data.bin", 0)
print(f"                                        0: {total}")

#
for channel in range(4):
    total = count_channel_activations_fast("data.bin", channel)
    print(f"            {channel}: {total}")
```

Листинг 2: Пример использования count_channel_activations_fast

Производительность: Функция читает файл один раз и выполняет минимальное количество операций, что делает ее в 10-100 раз быстрее стандартного подхода для больших файлов.

2.2 get_channel_stats_fast

```
def get_channel_stats_fast(filename: str, channel_num: int) -> Tuple[
    int, Dict[int, int]]:
```

Листинг 3: Сигнатура функции get_channel_stats_fast

Назначение: Быстрый подсчет детальной статистики по каналу с разбивкой по ячейкам.

Параметры:

- `filename: str` – путь к файлу для анализа
- `channel_num: int` – номер канала (0-3)

Возвращаемое значение:

- `Tuple[int, Dict[int, int]]` – кортеж, содержащий:
  - `int` – общее количество срабатываний в канале
  - `Dict[int, int]` – словарь, где ключ – номер ячейки (0-15), значение – количество срабатываний

Пример использования:

```
total, stats = get_channel_stats_fast("data.bin", 0)
print(f"                                    : {total}")
print("                    :")
for cell, count in stats.items():
    print(f"                {cell}: {count}        ")

#
active_cells = [cell for cell, count in stats.items() if count > 10]
print(f"                                    10
                                : {active_cells}")
```

Листинг 4: Пример использования get_channel_stats_fast

2.3 count_multiple_channels_fast

```
def count_multiple_channels_fast(filename: str, channels: List[int]) ->
    Dict[int, int]:
```

Листинг 5: Сигнатура функции count_multiple_channels_fast

Назначение: Подсчет срабатываний для нескольких каналов одновременно за один проход по файлу.

Параметры:

- `filename: str` – путь к файлу для анализа
- `channels: List[int]` – список номеров каналов для подсчета

Возвращаемое значение:

- `Dict[int, int]` – словарь, где ключ – номер канала, значение – количество срабатываний

Пример использования:

```
stats = count_multiple_channels_fast("data.bin", [0, 1, 2])
for channel, count in stats.items():
    print(f"            {channel}: {count}                        ")

#
max_channel = max(stats, key=stats.get)
print(f"                                    : {max_channel} ({stats[
    max_channel]}                           )")
```

Листинг 6: Пример использования count_multiple_channels_fast

2.4 get_channel_activation_timeline_fast

```
def get_channel_activation_timeline_fast(filename: str, channel_num:
    int,
                                        cell_num: Optional[int] = None)
                                            -> List[int]:
```

Листинг 7: Сигнатура функции get_channel_activation_timeline_fast

Назначение: Получение временных меток срабатываний для канала или конкретной ячейки.

Параметры:

- `filename: str` – путь к файлу для анализа
- `channel_num: int` – номер канала (0-3)
- `cell_num: Optional[int]` – если указан, возвращаются только срабатывания конкретной ячейки (0-15)

Возвращаемое значение:

- `List[int]` – список временных меток срабатываний

Пример использования:

```
#                                                    0
timeline_all = get_channel_activation_timeline_fast("data.bin", 0)
print(f"                                                    0: {len(
    timeline_all)}")

#
timeline_cell = get_channel_activation_timeline_fast("data.bin", 0,
    cell_num=5)
print(f"              5                    {len(timeline_cell)}        ")
if timeline_cell:
    print(f"              5            : {[f'{t:04X}' for t in
        timeline_cell[:5]]}")
```

Листинг 8: Пример использования get_channel_activation_timeline_fast

3 Стандартные функции (полный парсинг)

3.1 parse_data_file

```
def parse_data_file(filename: str) -> List[Tuple[int, List[int]]]:
```

Листинг 9: Сигнатура функции parse_data_file

Назначение: Основная функция для полного парсинга бинарного файла с созданием списка всех событий.

Параметры:

- `filename: str` – путь к файлу для парсинга

Возвращаемое значение:

- `List[Tuple[int, List[int]]]` – список кортежей, где каждый кортеж содержит:
  - `int` – временная метка события
  - `List[int]` – список из 4 чисел (каналы 0-3), каждое число представляет 16 бит состояния ячеек

Пример использования:

```python
events = parse_data_file("data.bin")
print(f"                          : {len(events)}")

for timestamp, channels in events[:5]:
    print(f"           : {timestamp:04X},              : {[f'{c:04X}' for c in channels]}")
```

Листинг 10: Пример использования parse_data_file

3.2 get_events_by_timestamp

```python
def get_events_by_timestamp(events: List[Tuple[int, List[int]]],
                            timestamp: int) -> List[List[int]]:
```

Листинг 11: Сигнатура функции get_events_by_timestamp

Назначение: Возвращает все события для указанной временной метки.
Параметры:

- `events` – список событий из `parse_data_file`
- `timestamp: int` – временная метка для поиска

Возвращаемое значение:

- `List[List[int]]` – список состояний каналов для всех записей с указанной меткой

3.3 get_channel_statistics

```python
def get_channel_statistics(events: List[Tuple[int, List[int]]],
                           channel_num: int) -> Dict[int, int]:
```

Листинг 12: Сигнатура функции get_channel_statistics

Назначение: Возвращает статистику срабатываний для указанного канала по ячейкам на основе распарсенных данных.
Параметры:

- `events` – список событий из `parse_data_file`
- `channel_num: int` – номер канала (0-3)

Возвращаемое значение:

- `Dict[int, int]` – словарь номер_ячейки: количество_срабатываний

3.4 print_channel_statistics

```python
def print_channel_statistics(stats: Dict[int, int], channel_num: int)
    -> None:
```

Листинг 13: Сигнатура функции print_channel_statistics

Назначение: Форматированный вывод статистики по каналу.
Параметры:

- `stats` – статистика из `get_channel_statistics`
- `channel_num: int` – номер канала для отображения

Пример вывода:

```
Статистика для канала 0:
----------------------------------------
Всего событий: 1250

Срабатывания по ячейкам:
Ячейка  0:    145 раз (11.60%)
Ячейка  1:     89 раз (7.12%)
Ячейка  2:    234 раз (18.72%)
...
```

3.5 filter_events_by_time_range

```python
def filter_events_by_time_range(events: List[Tuple[int, List[int]]],
                                start_time: int, end_time: int) -> List[
                                    Tuple[int, List[int]]]:
```

Листинг 14: Сигнатура функции filter_events_by_time_range

Назначение: Фильтрует события по диапазону временных меток.
Параметры:

- `events` – список событий
- `start_time: int` – начальная временная метка (включительно)
- `end_time: int` – конечная временная метка (включительно)

Возвращаемое значение:

- `List[Tuple[int, List[int]]]` – отфильтрованный список событий

3.6 get_cell_activation_timeline

```python
def get_cell_activation_timeline(events: List[Tuple[int, List[int]]],
                                 channel_num: int, cell_num: int) ->
                                    List[int]:
```

Листинг 15: Сигнатура функции get_cell_activation_timeline

Назначение: Возвращает список временных меток, когда конкретная ячейка сработала.
Параметры:

- `events` – список событий
- `channel_num: int` – номер канала (0-3)
- `cell_num: int` – номер ячейки (0-15)

Возвращаемое значение:

- `List[int]` – список временных меток срабатываний ячейки

3.7 get_channel_heatmap

```
def get_channel_heatmap(events: List[Tuple[int, List[int]]],
                        channel_num: int) -> List[List[int]]:
```

Листинг 16: Сигнатура функции get_channel_heatmap

Назначение: Создает "тепловую карту"активности для канала по временным меткам.
Возвращаемое значение:

- `List[List[int]]` – список, где каждый элемент содержит:
  - `[0]` – временная метка
  - `[1]` – состояние ячеек канала (16-битное число)

4 Функция сравнения производительности

4.1 compare_performance

```
def compare_performance(filename: str, channel_num: int = 0) -> None:
```

Листинг 17: Сигнатура функции compare_performance

Назначение: Сравнивает производительность разных методов подсчета и выводит детальную статистику.
Параметры:

- `filename: str` – путь к файлу для анализа
- `channel_num: int` – номер канала для тестирования (по умолчанию 0)

Пример вывода:

```
============================================================
СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ (канал 0)
============================================================

1. Стандартный метод (полный парсинг):
   - Время парсинга: 2.345с
   - Время анализа: 0.123с
   - Всего: 15000 событий
   - Память: ~1.5 MB

2. Быстрый метод (только количество):
   - Время: 0.156с
   - Всего: 15000 событий
   - Память: минимально
   - Ускорение: 15.8x

3. Быстрый метод (с детализацией):
   - Время: 0.178с
   - Всего: 15000 событий
   - Ячеек активно: 12
```

5 Сравнение методов

Таблица 2: Сравнение стандартного и быстрого подходов

Характеристика	**Стандартный подход**	**Быстрый подход**
Скорость	Медленнее (полный парсинг)	Быстрее в 10-100 раз
Использование памяти	Хранит все события в памяти $O(n)$	Минимальное $O(1)$
Гибкость	Высокая (можно фильтровать, анализировать)	Низкая (только подсчет)
Детализация	Полная информация о каждом событии	Только агрегированная статистика
Лучшее применение	Анализ небольших файлов, фильтрация по времени, визуализация	Большие файлы, быстрая статистика, мониторинг

6 Константы и настройки

6.1 Формат упаковки данных

По умолчанию используется big-endian формат (`'>H'`). Для little-endian замените:

```
#                                        (big-endian)
channel_data = struct.unpack('>H', data[pos:pos+2])[0]

#         little-endian
channel_data = struct.unpack('<H', data[pos:pos+2])[0]

#

#         little-endian                                                  :
channel_data = (data[pos + 1] << 8) | data[pos]  # little-endian
```

Листинг 18: Изменение порядка байт

6.2 Размер записи

```
record_size = 16  #
```

Листинг 19: Константа размера записи

7 Полные примеры использования

7.1 Пример 1: Быстрый анализ большого файла

```python
def monitor_channels(filename: str):
    """
                                """

    print("                                                        :")
    print("-" * 40)

    #
    stats = count_multiple_channels_fast(filename, [0, 1, 2, 3])

    total = sum(stats.values())
    for channel, count in stats.items():
        percentage = (count / total * 100) if total > 0 else 0
        print(f"            {channel}: {count:6d}
                                ({percentage:5.2f}%)")

    #

    if stats:
        max_channel = max(stats, key=stats.get)
        print(f"\ n
                                        {max_channel}:")
        total, cell_stats = get_channel_stats_fast(filename,
            max_channel)

        for cell in range(16):
            count = cell_stats.get(cell, 0)
            if count > total * 0.1:  #                  >10%

                print(f"                {cell}: {count}        ")

#
monitor_channels("large_data.bin")
```

Листинг 20: Быстрый анализ для мониторинга

7.2 Пример 2: Детальный анализ с фильтрацией

```python
def detailed_analysis(filename: str):
    """
                            """

    #                               (
                                                                )
    events = parse_data_file(filename)

    #
    intervals = [(0x0000, 0x3FFF), (0x4000, 0x7FFF), (0x8000, 0xFFFF)]

    for start, end in intervals:
        filtered = filter_events_by_time_range(events, start, end)
```

```
        print(f"\n                    {start:04X}-{end:04X}: {len(filtered)}                ")

        if filtered:
            stats = get_channel_statistics(filtered, 0)
            print_channel_statistics(stats, 0)

    #
    cell_timeline = get_cell_activation_timeline(events, 0, 7)
    print(f"\n             7                                    : {[f'{t:04X}' for t in cell_timeline[:10]]}...")
```

Листинг 21: Детальный анализ с фильтрацией по времени

7.3 Пример 3: Сравнение производительности

```
def test_performance():
    """
                                                    """

    test_files = {
        "small.bin": 1000,      # ~1000
        "medium.bin": 10000,     # ~10000
        "large.bin": 100000,     # ~100000
    }

    results = []

    for filename, expected_records in test_files.items():
        print(f"\n                        {filename} (~{expected_records}                )")
        print("-" * 50)

        #
        start = time.time()
        total_fast = count_channel_activations_fast(filename, 0)
        fast_time = time.time() - start

        #
        start = time.time()
        events = parse_data_file(filename)
        parse_time = time.time() - start

        start = time.time()
        stats = get_channel_statistics(events, 0)
        standard_time = time.time() - start + parse_time

        speedup = standard_time / fast_time if fast_time > 0 else 0

        results.append({
            'file': filename,
            'records': len(events),
            'fast_time': fast_time,
```

```
            'standard_time': standard_time,
            'speedup': speedup
        })

    #
    print("\n" + "="*60)
    print("                                                    ")
    print("="*60)
    print(f"{'          ':<15} {'                  ':<10} {'                  (
         )':<12} {'                          (   )':<15} {'
                          ':<10}")
    print("-"*62)

    for r in results:
        print(f"{r['file']:<15} {r['records']:<10} {r['fast_time
            ']:<12.3f} "
              f"{r['standard_time']:<15.3f} {r['speedup']:<10.1f}x")
```

Листинг 22: Тестирование производительности

8 Рекомендации по использованию

1. **Для больших файлов (сотни МБ и больше)** используйте быстрые функции:
   - `count_channel_activations_fast()` – для общего количества
   - `get_channel_stats_fast()` – для детальной статистики
   - `count_multiple_channels_fast()` – для нескольких каналов
2. **Для небольших файлов (до 10 тыс. записей)** можно использовать стандартные функции, если нужна гибкость:
   - `parse_data_file()` + `filter_events_by_time_range()`
   - `get_cell_activation_timeline()` – для временных рядов
3. **Для мониторинга в реальном времени** используйте быстрые функции с периодическим опросом файла
4. **Для визуализации данных** используйте стандартные функции для получения полной картины
5. **При работе с little-endian форматом** не забудьте изменить порядок байт в функциях

9 Обработка ошибок

```
def safe_analyze(filename: str, channel: int):
    """
                    """
    try:
        #
```

```
        if not os.path.exists(filename):
            print(f"            :           {filename}
                ")
            return None

        #
        if os.path.getsize(filename) == 0:
            print("            :                   ")
            return None

        #
        if channel < 0 or channel > 3:
            print("            :
                                0       3")
            return None

        #
        total = count_channel_activations_fast(filename, channel)
        print(f"            {channel}: {total}                          "
            )
        return total

    except PermissionError:
        print(f"            :
                        {filename}")
    except Exception as e:
        print(f"                                        : {e}")

    return None
```

Листинг 23: Рекомендуемая обработка ошибок

10 Примечания

1. Все временные метки обрабатываются как 16-битные целые числа (0-65535)
2. Ячейки в каналах нумеруются от 0 до 15 (младший бит соответствует ячейке 0)
3. Функции ожидают корректный формат входных данных
4. При несоответствии маркеров запись пропускается автоматически
5. Для работы модуля требуется Python 3.6 или выше
6. Быстрые функции используют побитовые операции для максимальной производительности
7. При работе с очень большими файлами (несколько ГБ) рекомендуется читать файл по частям, а не целиком